

A zero-knowledge proof of power for a formalized paper

The zkPoP construction, instantiated on the Lean 4 formalization of *Neural Quadratic Forms* (arXiv:2608.13335v1)

Abstract

This document describes zkPoP — a “zero-knowledge proof of power” pipeline built and machine-checked in Lean 4 — and its instantiation on the formalization of arXiv:2608.13335v1 (*Neural Quadratic Forms: A Unified Minimal Model for Sudden Learning and Scaling Laws*) contained in the same repository. The pipeline arithmetizes a *type checker* rather than an execution trace: a term of a small typed object language is compiled, through a tag-constraint system and a Tseitin flattening, into a gate circuit over a field, and that circuit is in turn checked by a universal circuit that takes the inner circuit as a private witness. Commitments, a three-move protocol and a Reed–Solomon/Shamir shard layer complete the deployment.

The instantiation is a *ledger*: ten headline claims of the paper, each carried together with the Lean statement the repository proves and *with its proof*, so a ledger entry cannot be created without the mathematics behind it. Over the field $\mathbb{Q}$ the document’s four headline results are proved: the ledger is honest, every ledger entry runs the full zkPoP chain to acceptance by the universal verifier, one aggregated circuit certifies all ten entries at once and is satisfied, and the published artifact is hiding while still binding the prover. All statements are Lean 4 theorems in `RequestProject/ZkPoP/NQFSkill.lean`; the file contains no `sorry` and every headline theorem’s axiom closure is exactly Lean’s three standard axioms.

Contents

1	What is being proved, and about what	2
2	The zkPoP pipeline	2
2.1	Phase 0: the object language	2
2.2	Phases I–II: arithmetization and compilation	2
2.3	Phase III: commitment and protocol	3
2.4	Phase IV: self-hosting	3
2.5	Phase V: deployment	3
3	The instantiation: a ledger of claims	3
3.1	The capability terms	4
4	Certification, claim by claim	4
5	One circuit for the whole paper	5
6	The deployment layer at $\mathbb{Q}$	5
7	The certificate	6
8	What was checked, and how to re-check it	6

1 What is being proved, and about what

A “proof of power” is a proof that the prover *possesses* a capability, published in a form that a third party can check but that does not hand the capability over. The capability of interest here is mathematical: the repository contains a Lean 4 formalization of the theory of neural quadratic forms of arXiv:2608.13335v1, and we wish to publish a single, checkable object certifying that the whole body of claims has been established, without publishing the witnesses.

The construction has two halves, and it is worth separating them at the outset.

- The **zkPoP pipeline** (Section 2) is generic. It compiles a term of an object language into a constraint system, the constraint system into a gate circuit, the gate circuit into an instance of a universal circuit, and wraps the result in a commitment and a shard distribution. Every step is accompanied by a soundness and, where meaningful, a completeness theorem in Lean.
- The **instantiation** (Sections 3–7) applies that pipeline to this repository’s formalization of the paper. Its object is a ledger of claims, and its headline result is a single certificate theorem.

Remark 1.1 (Scope and honesty). What is machine-checked is exactly what is stated. The cryptographic layer is proved at the level of its algebraic content: perfect hiding of the Pedersen commitment, binding modulo the discrete-logarithm assumption, special soundness of the three-move protocol, Reed–Solomon reconstruction and Shamir privacy. No claim is made here about a concrete elliptic-curve instantiation, a hash function, a Fiat–Shamir transform, or the computational hardness of any problem: those live outside the formalized boundary. Likewise the “zero-knowledge” content that is proved is the *indistinguishability* statement of Theorem 6.1, not a simulator-based definition.

2 The zkPoP pipeline

2.1 Phase 0: the object language

The object language is a small typed expression language with types

$$T ::= \text{nat} \mid \text{bool} \mid \text{vec } n T,$$

terms built from literals, `tru`, `fls`, `add`, `mul`, `ite`, `vrep` (constant vector) and `vhead`, a type checker `Tm.infer` and an evaluator `Tm.eval`. Types are encoded as field elements by an injective tag

$$\text{tag}(\text{nat}) = 0, \quad \text{tag}(\text{bool}) = 1, \quad \text{tag}(\text{vec } n A) = 2 + \langle n, \text{tag}(A) \rangle,$$

$\langle \cdot, \cdot \rangle$ being the Cantor pairing. The two facts that make the encoding usable are `Ty.tag_injective` (distinct types get distinct tags, so the arithmetization cannot collapse two types onto one wire) and the *semantic bridge* `Tm.checker_sound`: if the checker accepts t at type T , then evaluation of t succeeds and returns a value that really inhabits T . Everything downstream certifies the checker; the bridge is what makes certifying the checker worth anything.

2.2 Phases I–II: arithmetization and compilation

Phase I attaches to a term one wire per subterm, holding the tag of that subterm’s inferred type, and imposes the local constraints: for `add` and `mul` both operand tags and the output tag must be 0; for `ite` the condition tag must be 1 and the two branch tags must agree. The

correspondence `typeSat_iff` says the constraint system accepts exactly what the checker accepts, and `typeSat_unique` rules out junk wire vectors.

Phase II moves to a field $\mathbb{F}$. A *polynomial system* is a list of syntactic polynomials, satisfied by a wire vector $v : \mathbb{N} \rightarrow \mathbb{F}$ when each evaluates to 0; a *gate circuit* is a list of `const/add/mul/zero` gates. The Tseitin-style compiler `flatSys` consumes fresh scratch wires from an index n onwards, and `flatSys_sat_iff` states that compilation preserves satisfiability in both directions: a system is satisfiable by v iff its compilation is satisfiable by an extension of v agreeing with v below n . A catalog of gadgets — zero-testing with an inverse hint, multiplexers, Horner evaluation, knowledge of a root, and Merkle authentication paths — comes with individual soundness theorems and with `catalog_flat_faithful`, which transports them through the compiler.

The field-level version of the checker, `tySysF`, is the arithmetization used by the instantiation: one wire per subterm addressed by an injective Gödel code `Tm.code`, sound (`tySysF_sound`) and complete (`tySysF_complete`) for the base ($\mathbb{N}/\text{bool}$) fragment of the language, the canonical witness being the assignment that puts each subterm’s tag on its own wire.

2.3 Phase III: commitment and protocol

The commitment is Pedersen’s, $\text{com}(m, r) = m \cdot g + r \cdot h$ in a module G over a field R . Two theorems fix its behaviour: *perfect hiding*, i.e. for every m' there is a *unique* r' with $\text{com}(m, r) = \text{com}(m', r')$ whenever $r \mapsto r \cdot h$ is bijective; and *binding*, i.e. a double opening $\text{com}(m, r) = \text{com}(m', r')$ with $m \neq m'$ yields $g = (m - m')^{-1}(r' - r) \cdot h$, an explicit discrete logarithm of g base h . The three-move protocol on top of it is complete (`Schnorr.complete`) and specially sound: two accepting transcripts sharing an announcement determine an opening, computed by an explicit extractor `Schnorr.extract`.

2.4 Phase IV: self-hosting

The universal system `univSys` is the verifier written as a circuit that takes the inner circuit as a private witness, so that a proof about a circuit is itself a circuit statement (`univSys_sat_iff`, `univ_two_level`, `univ_self_hosting`). Library aggregation, `library_checked`, is what makes the instantiation of Section 5 possible:

Theorem 2.1 (Library aggregation). *For a list `lib` of gate circuits over $\mathbb{F}$, the aggregated circuit `libraryCircuit(lib)` is satisfied by $V : \mathbb{N} \rightarrow \mathbb{F}$ if and only if for every $k < |\text{lib}|$ the circuit `lib[k]` is satisfied by $w \mapsto V(\langle k, w \rangle)$: each member lives on its own block of wires.*

2.5 Phase V: deployment

The artifact is split over carriers as evaluations of a polynomial. Durability is Reed–Solomon decoding (`shards_recover`: any N shards of an artifact of degree $< N$ reconstruct it, and `shards_unique`: the reconstruction is unique); privacy is Shamir’s (`shards_hide_payload`: for any y there is a polynomial matching the given shards on s and taking the value y at a reserved index $o \notin s$).

3 The instantiation: a ledger of claims

The instantiation lives in `RequestProject/ZkPoP/NQFSkill.lean` (336 lines). Its central definition binds the object language to the mathematics.

Definition 3.1 (Claim). A `Claim` consists of: the number of a theorem in arXiv:2608.13335v1; a label; a *capability term* of the object language; the term’s type; a proof that the term lies in the base fragment; a proof that the object-language checker accepts the term at that type; and — the point of the construction — a *statement* : `Prop` together with a proof `proved` : `statement`.

Thm	claim	Lean theorem	digest
1	neural quadratic form	NQF.neural_quadratic_form	50
2	SGD closes on the order parameters	NQF.step_order_parameters	6
2	equal order parameters $\Rightarrow$ equal predictions	NQF.predictions_agree	156
3	width compression preserves the trajectory	NQF.compressed_predictions_agree	12
4	off-diagonal modes are slaved	NQF.offDiag_unique	20
5	orthogonal features: closed-form flow	NQF.orthogonal_features_solution	30
6	proportional samples: closed-form flow	NQF.proportional_samples_solution	42
7	orthogonal samples: closed-form flow	NQF.orthogonal_samples_solution	56
–	saddle-to-saddle: ignition gaps diverge	NQF.saddle_to_saddle	72
–	staircase loss and its power-law tail	NQF.limitLoss_powerLaw	90

Table 1: The ledger `paperLedger`. The digest is the value of the entry’s capability term under the object-language evaluator; for the generic term `capabilityTm(i)` it is $i+i^2$, and for the Theorem-1 entry it is the value of the paper’s quadratic form on an explicit configuration (Theorem 3.2).

Because `proved` is a field, a ledger entry cannot be constructed without a Lean proof of the mathematics it advertises. In `paperLedger` that field is filled in, in every case, by an already existing theorem of this repository’s formalization of the paper.

3.1 The capability terms

Each entry carries a term of the object language, checked at type `nat`. The generic term is

$$\text{capabilityTm}(i) = \text{ite } \text{tru } (i + i \cdot i) \ 0,$$

whose type-checking exercises the conditional, addition and multiplication rules simultaneously and whose value is the digest $i+i^2$. The first entry’s term is more than a placeholder: it computes, inside the object language, the paper’s quadratic form $\sum_i \text{Tr}[w_i w_i^\top A]$ on an explicit configuration, and this is proved to agree with the real-number computation performed in the formalization.

Theorem 3.2 (`traceTm_eval_eq_trace`). *Let $W = ((1,2),(3,4))$ and $A = \text{diag}(1,2)$, as the Lean definitions `Wex` and `Aex`. Then, in $\mathbb{R}$,*

$$50 = \sum_i \text{Tr}[w_i w_i^\top A],$$

and 50 is exactly the value returned by the object-language evaluator on the Theorem-1 capability term `traceTm`.

This is the binding between the two sides of the construction: the number the circuit certifies is the number the mathematics produces. The structure matrix `Aex` is also proved symmetric, as Theorem 1 of the paper requires.

4 Certification, claim by claim

Theorem 4.1 (`ledger_claims_hold`). *Every claim in `paperLedger` is a theorem: $\forall c \in \text{paperLedger}, c.\text{statement}$*

The proof is $\lambda c _ \mapsto c.\text{proved}$ — the content is not in the proof but in the construction of the ledger, since building it required supplying, for each entry, an actual project theorem of the advertised statement.

Theorem 4.2 (`ledger_certified`). *For every $c \in \text{paperLedger}$, over the field $\mathbb{Q}$:*

1. *the evaluator returns a value of the claimed type, $\exists \text{val}, \text{Tm.eval } c.\text{term} = \text{some val} \wedge \text{Val.hasTy val } c.\text{ty}$;*

2. the $\mathbb{N}$ -level tag system is satisfied: `typeSat c.term tag(c.ty)`;
3. the field-level system is satisfied by some $w : \mathbb{N} \rightarrow \mathbb{Q}$, with the term's own wire carrying `tag(c.ty)`;
4. there are n and an extension W of w below n satisfying the Tseitin compilation `flatSys(tySysFc.term) n`; and
5. the universal, self-hosting verifier accepts that compiled circuit: `univSys(flatSys(tySysFc.term) n)` is satisfied by W .

Chain (1)–(5) is the whole pipeline run on one entry, and it is obtained by instantiating the generic `end_to_end_recursive` at the entry's two structural fields (base fragment, checker acceptance). The converse direction is available generically as `end_to_end_sound`: a satisfying assignment of the compiled circuit really exhibits a typing derivation, hence a well-typed evaluation.

5 One circuit for the whole paper

For each entry, `claimBound` picks a wire bound for its constraint system, `claimCircuit` is the compiled circuit at that bound, and `claimWitness` a satisfying assignment (the canonical tag assignment, extended by the compiler). The aggregated object is

`paperCircuit = libraryCircuit(paperLedger.map claimCircuit),`

and `paperWitness` is the assignment placing the k -th entry's witness on the wire block $\langle k, \cdot \rangle$.

Theorem 5.1 (`paper_library_checked`). *`paperCircuit` is satisfied by `paperWitness`.*

By Theorem 2.1 this is equivalent to the simultaneous satisfaction of all ten compiled circuits, each on its own block: a single circuit over $\mathbb{Q}$ whose satisfaction certifies the type-correctness of every capability term in the ledger at once.

6 The deployment layer at $\mathbb{Q}$

The generic Phase-III and Phase-V results are instantiated at $R = G = \mathbb{Q}$ with the second generator $h = 1$, for which $r \mapsto r \cdot h$ is the identity and in particular bijective (`rat_smul_one_bijective`).

Theorem 6.1 (`paper_artifact_hides_witness`). *For all $g, m, m', r \in \mathbb{Q}$, a finite carrier set $s \subseteq \mathbb{Q}$, a reserved index $o \notin s$, shares shares and candidate payloads y, y' : there is a unique r' with $\text{com}(m, r) = \text{com}(m', r')$, and there are polynomials p, p' of degree $< |s| + 1$ agreeing on every carrier in s with $p(o) = y$ and $p'(o) = y'$.*

So the published data — the commitment together with any $|s|$ shards — is consistent with every witness and with every payload: it leaks nothing about either.

Theorem 6.2 (`paper_extractable`). *If two accepting transcripts share an announcement and have distinct challenges $c_1 \neq c_2$, then the commitment equals $\text{com}(\text{extract}(c_1, z_1, c_2, z_2), \text{extract}(c_1, w_1, c_2, w_2))$ and any two distinct openings of one commitment yield the discrete logarithm $g = (m - m')^{-1}(r' - r) \cdot h$.*

Theorem 6.3 (`paper_complete`, `paper_shards_recover`). *An honest prover always convinces the verifier, and the extractor then returns exactly the committed message and randomness. Any N shards of an artifact of degree $< N$ reconstruct it exactly.*

Hiding and binding are not in tension: hiding is unconditional, binding is conditional on the hardness of the discrete logarithm, which is where the assumption sits and is stated as an explicit implication rather than assumed.

7 The certificate

Theorem 7.1 (`nqf_zkpop_certificate`). *For all $g, m, m', r \in \mathbb{Q}$, a carrier set s , a reserved index $o \notin s$, shares and payloads y, y' , the conjunction of the four layers holds:*

1. honesty: every claim of `paperLedger` is a proved theorem of the project;
2. certification: every capability term satisfies the chain of Theorem 4.2, ending in acceptance by the universal self-hosting verifier;
3. aggregation: `paperCircuit` is satisfied by `paperWitness`;
4. privacy: the published artifact is consistent with every witness and every payload, as in Theorem 6.1.

8 What was checked, and how to re-check it

The instantiation is the Lean module `RequestProject/ZkPoP/NQFSkill.lean`, which imports the `zkPoP` pipeline (`RequestProject/ZkPoP/Skill.lean` and its dependencies) and the paper modules `Expansion`, `Master`, `Compression`, `Regimes`, `Solutions`, `Staircase` and `PowerLawLoss`. It contains no `sorry`. The aggregator `RequestProject/ZkPoP.lean` carries the audit: a `#print axioms` command for each headline theorem of every phase, now including Phase VI, the instantiation described here. Each reports exactly

`[propext, Classical.choice, Quot.sound],`

Lean’s three standard axioms, and no others.

To reproduce:

```
lake build RequestProject.ZkPoP
```

which elaborates the pipeline, the instantiation, and the audit commands; the axiom lines appear in the build log.

result	Lean name
the ledger is honest	<code>ZkPoP.Paper.ledger_claims_hold</code>
each entry runs the full chain	<code>ZkPoP.Paper.ledger_certified</code>
one circuit for all ten entries	<code>ZkPoP.Paper.paper_library_checked</code>
capability term = quadratic form	<code>ZkPoP.Paper.traceTm_eval_eq_trace</code>
the artifact hides the witness	<code>ZkPoP.Paper.paper_artifact_hides_witness</code>
the artifact binds the prover	<code>ZkPoP.Paper.paper_extractable</code>
honest prover, exact extraction	<code>ZkPoP.Paper.paper_complete</code>
durability of the shards	<code>ZkPoP.Paper.paper_shards_recover</code>
the certificate	<code>ZkPoP.Paper.nqf_zkpop_certificate</code>

Table 2: The headline results of the instantiation.

9 Limitations

- The capability terms are small. What the aggregated circuit certifies is the type-correctness of those terms, not, by itself, the mathematical content of the paper: the mathematical content enters through the `proved` field of the ledger, which is checked by Lean’s kernel rather than by the circuit. The two are tied together, for Theorem 1, by Theorem 3.2.

- The cryptography is algebraic, as described in the scope remark of Section 2: no concrete group, hash or Fiat–Shamir transform is formalized, and binding is a conditional statement about the discrete logarithm rather than a hardness assumption discharged.
- The instantiating field is $\mathbb{Q}$, chosen because it is a field with decidable equality in Lean and needs no primality side conditions. Nothing in the development is specific to it: every statement instantiated here is a specialization of a theorem proved for an arbitrary field.